

UMBRAMARKET S.R.L.

RAPPORTO FINALE

Vulnerability Assessment & Penetration Test

Golden Training Example — versione italiana

CLASSIFICAZIONE TRAINING / CONFIDENTIAL — tutti i sistemi, account e dati sono fittizi e limitati all’ambiente locale di laboratorio.

Campo	Valore
Cliente	UmbraMarket S.r.l. — scenario fittizio
Engagement	Vulnerability Assessment & Penetration Test
Tipo	Grey-box / web & API autenticato
Ambiente	Laboratorio locale Docker
Versione	1.0 — Final
Destinatari	Studenti ITS Cybersecurity VAPT

Documento didattico pensato per mostrare l’aspetto e la struttura di un deliverable professionale realmente consegnabile a un cliente.

1. Controllo del documento

Versione	Stato	Modifica
0.1	Bozza	Raccolta finding ed evidenze
0.5	Review	Correlazione attack chain e remediation
1.0	Finale	Versione didattica client-ready

2. Executive Summary

UmbraMarket ha richiesto un assessment controllato della piattaforma web locale con particolare attenzione a gestione degli input, separazione fra utenti, funzioni amministrative e informazioni esposte dallo staging.

L'assessment ha identificato cinque debolezze tecniche rilevanti. Il rischio principale è la mancata autorizzazione object-level sull'API ordini: un customer autenticato può richiedere l'ordine di un altro customer modificando il solo identificatore dell'oggetto. È stata inoltre verificata una failure di autorizzazione verticale: un customer può invocare una funzione amministrativa. Lo staging espone informazioni che facilitano la scoperta di tale funzione. Sono inoltre presenti una SQL Injection nella ricerca prodotti e una Reflected XSS nella funzione di greeting.

Le debolezze consentono movimenti applicativi orizzontali — customer verso dati di un altro customer — e verticali — customer verso una funzione amministrativa. Non è stato invece dimostrato movimento laterale di rete, accesso al sistema operativo, persistence, compromissione completa del database o account takeover.

PRIORITÀ Applicare autorizzazione server-side per oggetti e funzioni, parametrizzare le query SQL, ripristinare l'output encoding contestuale e rimuovere artefatti non necessari dallo staging.

3. Vista esecutiva del rischio

ID	Finding	Severità	Stato	Evidenza
F-01	Accesso all'ordine di un altro customer	ALTA	CONFERMATO	E-01 / E-02 / E-03
F-02	Customer invoca statistiche amministrative	MEDIA	CONFERMATO	E-04
F-03	SQL Injection nella ricerca prodotti	ALTA	CONFERMATO	E-06
F-04	Reflected Cross-Site Scripting nel greeting	MEDIA	CONFERMATO	E-07
F-05	Information disclosure dello staging	BASSA	CONFERMATO	E-05

Le severity sono qualitative e contestualizzate al laboratorio. In un assessment reale, se richiesto, il team dovrebbe riportare CVSS con versione, vettore e motivazione.

4. Obiettivi dell'engagement

4.1 Domande di business

- un cliente normale può accedere ai dati di un altro cliente?
- un cliente normale può raggiungere funzioni amministrative?
- informazioni apparentemente innocue possono facilitare un attacco successivo?
- input controllato dall'utente può alterare query o contenuto eseguito nel browser?
- fino a dove arriva realmente ciascuna catena d'attacco?
- quali controlli devono essere corretti e come verrà verificata la remediation?

4.2 Obiettivi di sicurezza

- Mappare la superficie HTTP autorizzata.
- Osservare sessioni, ruoli, oggetti ed endpoint.
- Validare manualmente i candidate finding.
- Usare una prova minima e reversibile.
- Distinguere impatto dimostrato da impatto potenziale.
- Definire remediation e criteri di retest riproducibili.

5. Scope

5.1 In scope

Asset	Funzione	Stato
127.0.0.1:5005	UmbraMarket Guided web app + API	IN SCOPE
127.0.0.1:8080	Admin staging	IN SCOPE
127.0.0.1:9090	File service	IN SCOPE

5.2 Out of scope

Asset	Motivo
127.0.0.1:3000	OWASP Juice Shop escluso dal Lab 13
LAN dello studente	Non autorizzata
Internet / sistemi esterni	Non autorizzati
Altri host o porte	Non autorizzati

6. Rules of Engagement

Attività consentite

- Browser e Developer Tools
- curl
- Burp Suite Community oppure OWASP ZAP
- Nmap sulle sole porte autorizzate
- Login con account didattici
- Modifica controllata di parametri e object ID
- Content discovery non distruttiva
- PoC manuale minima e raccolta request/response

Attività non consentite

- Brute force
- Denial of service
- Distruzione o modifica dei dati
- Persistence
- Mass enumeration / estrazione massiva
- Callback verso sistemi esterni
- Attacchi a persone o sistemi reali
- Test oltre lo scope

STOP CONDITION Ottenuta una singola evidenza sufficiente a dimostrare il failure di sicurezza, il test si ferma.

7. Metodologia

```

SCOPE
↓
RECONNAISSANCE
↓
APPLICATION MAPPING
↓
BASELINE
↓
HYPOTHESIS
↓
CONTROLLED TEST
↓
EXPECTED vs OBSERVED
↓
EVIDENCE
↓
IMPACT
↓
ATTACK CHAIN / PIVOT ANALYSIS
↓
REMEDIATION
↓
RETEST

```

Stato evidenza	Significato
OBSERVED	Informazione osservata ma non ancora prova di vulnerabilità
VALIDATED	Comportamento di sicurezza verificato
TRAINING REFERENCE CAPTURE	Evidenza didattica da ricattare live
NOT DEMONSTRATED	Ipotesi senza prova sufficiente
OUT OF SCOPE	Non deve essere testato
STOPPED	Evidenza già sufficiente o limite RoE raggiunto

REGOLA DI QUALITÀ Ogni finding deve spiegare: cosa abbiamo testato, cosa doveva succedere, cosa è successo, quale evidenza lo dimostra, quale impatto è stato realmente dimostrato e cosa non possiamo affermare.

8. Ambiente e strumenti

```

cd labs/platform
docker compose pull
docker compose up -d --build
./scripts/check.sh

```

Tool	Uso
Browser	Navigazione, comportamento applicativo, PoC XSS
DevTools	Network, cookie, request/response, endpoint mapping
curl	Riproduzione minimale e scriptabile
Burp Suite Community	Proxy, HTTP History, Repeater, confronto richieste
OWASP ZAP	Alternativa a Burp per proxy e request manipulation
Nmap	Ricognizione dei soli servizi locali autorizzati

Kali Linux non è un requisito: target e strumenti possono essere eseguiti sullo stesso PC dello studente.

9. Reconnaissance e superficie d'attacco

```
nmap -sV -p 5005,8080,9090 127.0.0.1
```

Porta	Superficie	Valutazione
5005	Web app + REST API	Testata
8080	Admin staging statico	Testata
9090	File service	Enumerazione didattica
3000	Juice Shop	OUT OF SCOPE

La presenza di una porta aperta non costituisce, da sola, una vulnerabilità.

10. Condizioni iniziali e autenticazione

Per F-01 e F-02 non è stato dimostrato alcun authentication bypass. Il punto di partenza è un normale account customer valido.

```
NORMAL CUSTOMER
↓
VALID LOGIN
↓
AUTHENTICATED SESSION
↓
TEST AUTHORIZATION BOUNDARIES
```

INTERPRETAZIONE CORRETTA Il server conosce l'identità dell'utente, ma prende una decisione di autorizzazione errata. Non significa che un attaccante anonimo abbia compromesso il sistema.

11. Sintesi delle catene d'attacco

AC-01 — Movimento orizzontale applicativo

```
Alice customer
↓
Alice order baseline
↓
request Bob order ID
↓
ownership check missing
↓
Bob order returned
```

Capability ottenuta: lettura cross-user di un singolo ordine dimostrato. Non dimostrato: tutti gli ordini, scrittura, account takeover o DB dump.

AC-02 — Staging discovery → boundary verticale

```
Admin staging :8080
↓
```

```

robots.txt
↓
backup / JavaScript metadata
↓
/api/admin/stats discovered
↓
Alice customer session
↓
role check missing
↓
administrative statistics returned

```

AC-03 — Manipolazione della query SQL

```

public product search
↓
attacker-controlled q
↓
SQL string concatenation
↓
predicate altered
↓
unexpected result set

```

AC-04 — Contesto di esecuzione browser

```

/greet?name=
↓
untrusted input
↓
response HTML without output encoding
↓
browser interprets active markup
↓
benign JavaScript proof executes

```

MOVIMENTO LATERALE Nessuna evidenza dimostra movimento laterale di rete. In F-01 si parla di movimento orizzontale applicativo fra oggetti/identità customer.

F-01 — Un customer può leggere l'ordine di un altro customer

SEVERITÀ	CATEGORIA	ASSET	STATO
ALTA	BOLA / IDOR / Authorization	GET /api/orders/{id}	CONFERMATO

Business rule attesa: un customer deve poter accedere esclusivamente ai propri ordini. Il server deve collegare identità autenticata, oggetto richiesto e ownership prima di restituire i dati.

Precondizioni

- Account customer valido.
- Sessione autenticata.
- Identificatore di un ordine appartenente a un altro customer.

Validazione ed evidenze

E-01 stabilisce un ordine legittimo di Alice; E-02 stabilisce che l'ordine 1002 appartiene al contesto Bob; E-03 mantiene la sessione Alice e cambia soltanto l'ID richiesto.

```
E-01 – Alice baseline
GET /api/orders/1001 HTTP/1.1
Host: 127.0.0.1:5005
Cookie: session=<redacted-alice-session>
```

```
E-02 – Bob baseline
GET /api/orders/1002 HTTP/1.1
Host: 127.0.0.1:5005
Cookie: session=<redacted-bob-session>
```

```
E-03 – Cross-user test
GET /api/orders/1002 HTTP/1.1
Host: 127.0.0.1:5005
Cookie: session=<redacted-alice-session>
```

Expected	Observed
403/404 senza dati Bob	HTTP 200 con ordine 1002 e indirizzo di spedizione Bob

Riproduzione con Browser / Burp

1. Login come Alice.
2. Aprire Dashboard e osservare una richiesta ad un ordine Alice.
3. Inviare la request a Burp Repeater.
4. Sostituire l'ID con 1002 mantenendo la sessione Alice.
5. Inviare e confrontare con la baseline Bob.
6. Fermarsi dopo la prima prova positiva.

Impatto dimostrato

Un customer può leggere dati di ordine appartenenti a un altro customer; nel dataset lab è incluso anche l'indirizzo di spedizione.

```
Alice identity → Alice-authorized object → change object identifier → Bob-owned object
```

Non dimostrato

- Accesso a tutti gli ordini
- Enumerazione massiva
- Modifica o cancellazione
- Compromissione database
- Takeover Bob
- Shell sul server

Root cause

Il backend verifica la presenza della sessione, ma recupera l'ordine per ID senza includere l'identità autenticata nella decisione di autorizzazione.

Remediation

Applicare ownership check server-side su ogni accesso all'oggetto. L'identità deve derivare dalla sessione verificata, non da parametri controllabili dal client.

```
SELECT ...
FROM orders
WHERE id = ?
AND user_id = ?
```

- Policy di authorization centralizzata.

- Negative authorization tests.
- Test Alice→Bob e Bob→Alice in CI.
- Review di ogni endpoint object-based.
- UUID solo come defense-in-depth: non sostituiscono authorization.

Retest

```
Alice → Alice order = ALLOW  
Alice → Bob order   = DENY  
Bob   → Bob order   = ALLOW
```

PASS: 403/404 e nessun dato Bob. FAIL: qualunque dato Bob viene ancora restituito ad Alice.

F-02 — Un customer può invocare statistiche amministrative

SEVERITÀ	CATEGORIA	ASSET	STATO
MEDIA	Broken Function Level Authorization	GET /api/admin/stats	CONFERMATO

La vulnerabilità non è la semplice discoverability dell'endpoint: il failure avviene quando il server consente ad un ruolo customer di invocare una funzione amministrativa.

Discovery

```
:8080/robots.txt → /backup/ → /backup/appsettings.old → /assets/app.js → /api/admin/stats
```

Validazione

```
GET /api/admin/stats HTTP/1.1
Host: 127.0.0.1:5005
Cookie: session=<redacted-alice-session>
```

```
HTTP/1.1 200 OK
{
  "users": 3,
  "orders": 3,
  "revenue": 367.9
}
```

Expected	Observed
403 Forbidden / deny	HTTP 200 con dati aggregati amministrativi

Impatto dimostrato

Un customer può ottenere dati aggregati riservati ad una funzione amministrativa.

```
customer → authenticated session → admin function → missing role check → privileged information
```

Non dimostrato

- Modifica utenti
- Cambio ruoli
- Scrittura configurazione
- Creazione admin
- Remote code execution
- Server takeover

Root cause e remediation

È presente authentication, ma manca la verifica server-side del ruolo/permesso richiesto dalla funzione. Applicare deny-by-default, policy di autorizzazione esplicite sull'intero namespace admin e test negativi customer→admin.

Retest

```
customer → /api/admin/stats = DENY
admin → /api/admin/stats = ALLOW
anonimo → /api/admin/stats = DENY
```

F-03 — SQL Injection nella ricerca prodotti

SEVERITÀ	CATEGORIA	ASSET	STATO
----------	-----------	-------	-------

SEVERITÀ	CATEGORIA	ASSET	STATO
ALTA	Injection / SQL Injection	GET /search?q=	CONFERMATO

Input non fidato viene concatenato nella query invece di essere passato come parametro dati.

Prova minima

Baseline: /search?q=Umbra

Input controllato:

%' OR 1=1 --

Riproduzione:

curl -G --data-urlencode "q=%' OR 1=1 -- " http://127.0.0.1:5005/search

Expected	Observed
Il valore viene trattato come semplice termine di ricerca	Il predicato viene alterato e compaiono tutti i prodotti seed

Impatto dimostrato

L'input dell'utente può modificare la logica della query eseguita dal database.

Potenziale — NON DIMOSTRATO

- Dump tabella users
- Estrazione password
- Scrittura dati
- File read/write
- Command execution
- DB administrator compromise

Remediation

Usare query parametrizzate e binding del driver. L'input filtering non sostituisce la parametrizzazione. Applicare inoltre least privilege dell'account DB, gestione errori sicura e test automatici per input speciali.

```
search term = q
LIKE parameter = "%" + q + "%"
```

```
SQL:
WHERE name LIKE ?
```

Retest

PASS: il payload viene cercato come testo e non altera il result set. **FAIL:** l'input modifica ancora la semantica SQL.

F-04 — Reflected Cross-Site Scripting nel greeting

SEVERITÀ	CATEGORIA	ASSET	STATO
MEDIA	Cross-Site Scripting / Output Encoding	GET /greet?name=	CONFERMATO

Il parametro name viene riflesso nella risposta come markup attivo invece di essere codificato come testo.

Prova minima

Baseline: /greet?name=Alice

PoC innocua:
 <script>alert(1)</script>

Costruzione richiesta:
 curl -G --data-urlencode "name=<script>alert(1)</script>" http://127.0.0.1:5005/greet

Expected	Observed
Ciao, <script>alert(1)</script>!	Markup attivo; il browser esegue alert(1)

Impatto dimostrato

È possibile eseguire JavaScript nell'origine web quando una vittima apre una richiesta appositamente costruita.

Potenziale — NON DIMOSTRATO

- Furto cookie
- Takeover account
- Azioni come admin
- Credential harvesting
- Persistence
- Callback esterni

Remediation

- Non promuovere input utente a trusted markup.
- Mantenere auto-escaping del template engine.
- Applicare contextual output encoding.
- Evitare innerHTML / HTML injection lato client.
- CSP come defense-in-depth, non come sostituto dell'encoding.

Retest

PASS: il payload compare solo come testo e nessun JavaScript viene eseguito. **FAIL:** markup/script controllato viene ancora interpretato.

F-05 — Lo staging espone informazioni utili alla discovery

SEVERITÀ	CATEGORIA	ASSET	STATO
BASSA	Information Disclosure / Misconfiguration	127.0.0.1:8080	CONFERMATO

Lo staging rende pubblici artefatti e metadata che riducono il lavoro necessario per individuare endpoint applicativi privilegiati.

Evidenze

```
robots.txt
Disallow: /backup/
Disallow: /draft/

backup/appsettings.old
environment=staging
api_base=http://app.umbraMarket.test:5005/api
log_level=debug
feature_admin_v2=false

assets/app.js
```

```
apiBase: http://127.0.0.1:5005/api  
statsEndpoint: /admin/stats  
environment: staging
```

Riproduzione

```
curl http://127.0.0.1:8080/robots.txt  
curl http://127.0.0.1:8080/backup/appsettings.old  
curl http://127.0.0.1:8080/assets/app.js
```

Impatto dimostrato

Le informazioni di implementazione consentono di individuare più rapidamente l'endpoint amministrativo validato in F-02.

DISTINZIONE IMPORTANTE Conoscere `/api/admin/stats` non dovrebbe essere sufficiente per comprometterlo. Nascondere l'endpoint non risolve la mancanza di authorization di F-02.

Remediation

- Rimuovere backup e draft dal web root.
- Non usare robots.txt per proteggere contenuti sensibili.
- Separare build artifacts da file operativi.
- Limitare l'accesso allo staging dove appropriato.
- Minimizzare metadata pubblici non necessari.
- Mantenere authorization indipendente dalla discoverability.

17. Correlazione dei percorsi d'attacco

17.1 Correlazione confermata A

```
E-05 Information Disclosure
↓
admin endpoint discovered
↓
E-04 / F-02
↓
customer crosses vertical authorization boundary
```

17.2 Correlazione confermata B

```
normal customer access
↓
own-order baseline
↓
object ID manipulation
↓
F-01 BOLA / IDOR
↓
other-customer order disclosed
```

17.3 SQLi — ipotesi non validata oltre la prova minima

```
F-03 SQL Injection
↓
hypothesis: other DB data
↓
NOT TESTED — STOP CONDITION
```

17.4 XSS — ipotesi non validata oltre la PoC

```
F-04 XSS
↓
victim execution context
↓
hypothesis: authenticated action
↓
NOT TESTED
```

18. Cosa può fare realmente un utente normale

```
valid customer credentials
→ POST /login
→ authenticated session cookie

customer
├─ own dashboard           EXPECTED
├─ own orders              EXPECTED
├─ other customer's order  F-01
└─ administrative statistics F-02
```

Non esiste evidenza che il customer ottenga una shell, diventi admin, modifichi ordini o raggiunga altri host.

19. Triage scanner e non-finding

N-01 — Possibile RCE nginx segnalata dallo scanner

Stato: NOT DEMONSTRATED. Un banner/version fingerprint non dimostra da solo una vulnerabilità, i prerequisiti, l'exploitability, la code execution o l'impatto.

O-01 — CSP assente

Osservazione di hardening. L'assenza di CSP riduce la defense in depth ma non sostituisce la root cause della XSS e non dimostra, da sola, compromissione.

TEACHING RULE scanner severity $\neq$ validated risk. Un candidate finding diventa reportabile solo dopo validazione manuale ed evidenza.

20. Registro evidenze

ID	Evidenza	Tipo	Supporta
E-01	Alice own-order baseline	LAB-VALIDATED	F-01
E-02	Bob own-order baseline	LAB-VALIDATED	F-01
E-03	Alice legge ordine Bob	LAB-VALIDATED	F-01
E-04	Customer invoca admin stats	LAB-VALIDATED	F-02
E-05	Staging content discovery	LAB-REPRODUCIBLE	F-05 / F-02
E-06	Minimal SQL query manipulation	TRAINING REFERENCE	F-03
E-07	Minimal reflected XSS proof	TRAINING REFERENCE	F-04
S-001	Scanner possible RCE	CANDIDATE ONLY	N-01
S-003	Missing CSP	OBSERVATION	O-01

E-06 ed E-07 vanno ricatturate live durante un'esercitazione reale. Un buon report non inventa timestamp, header, screenshot o risposte non catturate.

21. Roadmap di remediation

Priorità	Azione	Finding	Owner suggerito
P1	Enforce object ownership server-side	F-01	Backend
P1	Enforce role/permission on admin functions	F-02	Backend / IAM
P1	Parametrizzare le query SQL	F-03	Backend / Data
P2	Ripristinare contextual output encoding	F-04	Backend / Frontend
P2	Aggiungere negative security tests alla CI	F-01..F-04	Engineering / QA
P2	Rimuovere backup/draft dallo staging	F-05	DevOps
P3	Introdurre CSP defense in depth	O-01	Frontend / Platform
P3	Ritestare periodicamente lo staging	F-05	AppSec / DevOps

21.1 Temi strutturali

Authorization policy

- object-level
- function-level
- deny-by-default
- test negativi

Untrusted input handling

- SQL parameters
- contextual output encoding
- safe defaults del framework

Environment hygiene

- niente backup nel web root
- staging minimizzato
- metadata pubblici solo se necessari

22. Matrice di retest

Test	Expected PASS
Alice → Alice order	200, dati Alice
Alice → Bob order	403/404, zero dati Bob
Bob → Bob order	200, dati Bob
Customer → admin stats	403/404
Admin → admin stats	200
SQLi test input	Trattato come testo, nessuna alterazione query
XSS controlled input	Encoded, nessuna esecuzione
/backup/appsettings.old	Non disponibile
Admin JS	Nessun metadata non necessario

23. Appendice operativa — riproduzione con i tool

NOTA Questa sezione è intenzionalmente operativa perché il documento è un training sample. In un report reale non si pubblicano password o cookie validi.

23.1 Sessione Alice con curl

```
curl -i -c alice.cookies -X POST -d "username=alice&password=Alice123!" http://127.0.0.1:5005/login
curl -b alice.cookies http://127.0.0.1:5005/api/me
```

23.2 Sessione Bob

```
curl -i -c bob.cookies -X POST -d "username=bob&password=Bob123!" http://127.0.0.1:5005/login
```

23.3 Baseline ownership

```
curl -b alice.cookies http://127.0.0.1:5005/api/orders/1001
curl -b bob.cookies http://127.0.0.1:5005/api/orders/1002
```

23.4 Prova cross-user minima

```
curl -b alice.cookies http://127.0.0.1:5005/api/orders/1002
```

STOP dopo la singola risposta positiva.

23.5 Funzione admin

```
curl -b alice.cookies http://127.0.0.1:5005/api/admin/stats
```

23.6 Staging discovery

```
curl http://127.0.0.1:8080/robots.txt
curl http://127.0.0.1:8080/backup/appsettings.old
curl http://127.0.0.1:8080/assets/app.js
```

23.7 SQL Injection — prova minima

```
curl -G --data-urlencode "q=%' OR 1=1 -- " http://127.0.0.1:5005/search
```

Fermarsi dopo aver confermato la query manipulation. Non eseguire dump di tabelle non necessari.

23.8 XSS — prova minima

```
curl -G --data-urlencode "name=<script>alert(1)</script>" http://127.0.0.1:5005/greet
```

La reflection è visibile con curl; l'esecuzione va validata nel browser usando solo alert(1) e fermandosi subito.

23.9 Workflow Burp Repeater

```
Proxy / HTTP History
→ select valid request
→ Send to Repeater
→ change ONE variable
→ Send
→ compare status/body
→ save request + response
```

Cambiare una sola variabile alla volta rende l'evidenza più forte perché isola il fattore che causa il comportamento.

24. Checklist di gestione delle evidenze

- Evidence ID univoco
- Target
- Identità/ruolo se rilevante
- Request completa con segreti redatti
- Response rilevante
- Expected behavior
- Observed behavior
- Interpretazione
- Stop condition
- Not demonstrated
- Collegamento al finding

NON INSERIRE NEL REPORT Cookie validi, password reali, API key, token, dati personali non necessari o dump massivi.

25. Cosa è stato dimostrato

- Un customer può leggere almeno un ordine appartenente a un altro customer.
- Un customer può raggiungere una funzione amministrativa di statistiche.
- Lo staging facilita la discovery dell'endpoint amministrativo.
- La search contiene una SQL Injection dimostrabile con query manipulation minima.
- Il greeting contiene una Reflected XSS dimostrabile con PoC browser innocua.

26. Cosa NON è stato dimostrato

- Anonymous full compromise
- Accesso a tutti gli ordini
- Modifica/cancellazione ordini
- Dump dell'intero database
- Credential theft
- Admin account takeover
- OS command execution
- Remote shell
- Persistence
- Network lateral movement
- Compromissione di altri host
- Malware execution
- Impatto su sistemi reali

27. Conclusioni

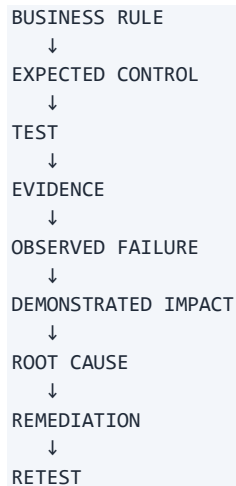
Il rischio principale di UmbraMarket non deriva da una singola tecnologia, ma dall'assenza di security boundary coerenti in punti diversi dell'applicazione.

F-01 dimostra un failure di authorization orizzontale: l'identità è correttamente autenticata ma l'ownership dell'oggetto non viene verificata. F-02 dimostra un failure verticale: un customer raggiunge una funzione amministrativa perché manca il controllo ruolo/permesso.

F-03 e F-04 mostrano due classiche conseguenze del trattamento non sicuro di input non fidato: l'input viene interpretato rispettivamente come parte di una query SQL e come markup/script browser. F-05 mostra come informazioni di staging apparentemente poco importanti possano diventare utili quando correlate con un'altra debolezza.

La remediation deve quindi essere affrontata per classi di controllo — authorization, input handling ed environment hygiene — e non soltanto correggendo i cinque URL osservati. Dopo le correzioni è raccomandato un retest focalizzato basato sulla matrice PASS/FAIL definita nel presente documento.

28. Debrief didattico — cosa rende questo un vero report VAPT



1. Da dove parte l'attaccante?
2. Quale preconditione serve?
3. Quale boundary viene attraversato?
4. Quale singola prova lo dimostra?
5. Cosa ha ottenuto realmente?
6. Quale passo successivo diventa plausibile?
7. Quel passo è stato testato o è solo ipotizzato?
8. Dove ci siamo fermati e perché?
9. Qual è la root cause?
10. Come dimostreremo che la fix funziona?

MESSAGGIO FINALE La differenza tra “ho trovato una vulnerabilità” e un deliverable professionale è la capacità di collegare evidenza, impatto, limiti, remediation e retest.

Appendice A — Credenziali didattiche

SOLO LABORATORIO LOCALE Non riutilizzare queste credenziali altrove.

```
alice / Alice123!  
bob   / Bob123!  
admin / Admin123!
```

Alice e Bob sono sufficienti per dimostrare i boundary customer. L'account admin è utile soprattutto nel retest del comportamento amministrativo legittimo.

Appendice B — Evidence files

```
reporting/evidence/  
├─ E-01-alice-own-order.txt  
├─ E-02-bob-own-order.txt  
├─ E-03-cross-user-order.txt  
├─ E-04-customer-admin-stats.txt  
├─ E-05-staging-information-disclosure.txt  
├─ E-06-sql-injection-minimal-proof.txt  
└─ E-07-reflected-xss-minimal-proof.txt
```

Appendice C — Uso in aula

1. Confrontare il finding degli studenti con F-01.
2. Evidenziare expected vs observed.
3. Far cercare ogni frase di impatto priva di Evidence ID.
4. Discutere confirmed vs plausible.
5. Confrontare authorization orizzontale e verticale.
6. Mostrare perché staging→admin è più utile di due finding isolati.
7. Far riscrivere un finding scanner-based usando solo fatti verificati.
8. Chiudere con remediation e retest, non con la PoC.